

Terraform Lifecycle: Complete Flow + Our Use Case + TF Files

This version adds the Terraform files involved in the lifecycle, the role of each file, and exactly which lifecycle phase reads or uses each file.

PART 1 - COMPLETE TERRAFORM LIFECYCLE

Phase	What exactly happens - 1 liner
Write configuration	You define desired infrastructure in .tf files using resources, modules, variables, outputs, data sources, and provider configuration.
terraform init	Terraform initializes the working directory, downloads required providers/modules, and initializes the backend.
terraform validate	Terraform checks that the configuration is syntactically and structurally valid.
terraform plan	Terraform evaluates configuration, variables, data sources, state, and remote infrastructure, builds dependencies, and calculates proposed changes.
terraform apply	Terraform executes the planned changes through provider APIs and updates state.
State management	Terraform stores the mapping between configuration resources and real infrastructure so future runs can calculate differences.
Change -> plan -> apply	After a .tf or variable change, Terraform recalculates the delta and applies only the necessary changes.
terraform destroy	Terraform calculates removal of managed resources and deletes them through provider APIs, respecting dependencies.

Key mental model: .tf files describe what you want; the provider knows how to communicate with GCP; plan calculates the difference; apply makes it real; state remembers what Terraform manages.

PART 2 - TERRAFORM FILES AND THEIR ROLES

Terraform does not require specific filenames for most .tf files. All .tf files in the same directory are loaded together. The names below are conventional because they make a project easier to understand.

Typical file	Role	Main lifecycle phase(s)
providers.tf	Configures required providers, provider versions, and provider settings such as GCP project/region.	init, validate, plan, apply, destroy
versions.tf	Declares required Terraform version and required provider versions.	init, validate
backend.tf	Defines where Terraform state is stored, for example a remote GCS backend.	init; state operations
main.tf	Usually contains the main resources/modules that describe the infrastructure to create/manage.	validate, plan, apply, destroy
variables.tf	Declares input variables, their types, defaults, and descriptions.	validate, plan, apply
terraform.tfvars	Supplies concrete values for variables; commonly used for environment-specific values.	plan, apply, destroy
.tfvars (dev/test/pr-.tfvars)	Environment-specific variable values. Often selected explicitly by CI/CD.	plan, apply, destroy
locals.tf	Defines calculated/reusable local values used by resources and modules.	validate, plan, apply, destroy
data.tf	Defines data sources used to read existing information from GCP or elsewhere rather than create it.	plan/apply; sometimes destroy evaluation
outputs.tf	Defines values Terraform exposes after apply, such as cluster name, endpoint, or IDs used by downstream automation.	plan, apply
module files	A module directory contains its own main.tf, variables.tf, outputs.tf, etc.; modules package reusable infrastructure.	init, validate, plan, apply, destroy
.terraform.lock.hcl	Locks provider dependency selections/checksums so provider installation is repeatable.	init
terraform.tfstate / remote state	Records Terraform's state for managed resources. In CI, state is normally remote rather than a local file.	plan, apply, destroy, state operations

Important: Files such as main.tf, variables.tf, and outputs.tf are conventions, not lifecycle commands. Terraform loads all .tf files in the module directory as one configuration.

PART 3 - WHICH FILE IS USED IN WHICH PHASE?

Lifecycle phase	Files / artifacts involved	What happens in this phase
terraform init	versions.tf, providers.tf, backend.tf, module source definitions, .terraform.lock.hcl	Terraform determines required providers/modules, installs compatible versions, and initializes the backend/state connection.
terraform validate	All loaded .tf files: main.tf, variables.tf, locals.tf, data.tf, outputs.tf, providers.tf, module files	Terraform parses and validates the complete configuration and its references.
terraform plan	All .tf files + variable values + data sources + state + provider	Terraform evaluates desired state, reads needed existing data, compares with state/remote objects, builds the dependency graph, and creates the plan.
terraform apply	The plan + all configuration + provider + state	Terraform sends create/update/replace operations through the provider and records results in state.
State update	terraform.tfstate or remote backend	Terraform persists resource IDs and attributes needed to manage those resources on later runs.
Future plan/apply	All relevant .tf files + variables + state	A configuration change causes Terraform to recalculate the delta; only affected resources are changed.
terraform destroy	All .tf files + variables + state + provider	Terraform calculates what managed infrastructure must be removed and calls provider APIs to destroy it.

One important nuance

Terraform does not execute main.tf first, then variables.tf, then outputs.tf as separate sequential stages. It loads the configuration from the directory as a whole. The lifecycle phase determines *what Terraform does with that configuration*.

PART 4 - OUR USE CASE: FILE-BY-FILE

Assume an ephemeral environment repository like this:

```
terraform/  
  providers.tf  
  versions.tf  
  backend.tf  
  main.tf  
  variables.tf  
  locals.tf  
  data.tf  
  outputs.tf  
  terraform.tfvars / pr.tfvars  
  .terraform.lock.hcl  
modules/  
  network/  
  gke/  
  cloudsql/  
  iam/
```

File	Role in our GKE + Cloud SQL use case	Lifecycle connection
providers.tf	Configures the Google provider so Terraform can communicate with GCP.	init discovers/installs it; plan/apply/destroy use it.
versions.tf	Pins/constraints Terraform and Google provider versions for consistent CI runs.	init uses these constraints.
backend.tf	Points Terraform state to the team's remote backend, typically a GCS bucket.	init initializes backend; later plan/apply/destroy read/write state.
main.tf	Composes modules/resources for the ephemeral environment.	Loaded during validate; drives plan/apply/destroy.
variables.tf	Defines inputs such as project_id, region, environment/PR name, DB sizing, GKE settings.	Used when plan/apply/destroy evaluate configuration.
pr.tfvars	Supplies PR-specific values such as environment name or resource settings.	Read when CI invokes plan/apply/destroy with that variable file.
locals.tf	Builds reusable names/tags such as pr-125, labels, or common settings.	Evaluated during plan/apply/destroy.
data.tf	Reads existing GCP information, for example project, network, or other resources managed elsewhere.	Usually evaluated during plan/apply when values are needed.
modules/gke	Contains reusable GKE resources such as cluster/node-pool configuration.	Initialized as module dependency; evaluated during plan/apply/destroy.
modules/cloudsql	Contains Cloud SQL PostgreSQL resources and settings.	Initialized as module dependency; evaluated during plan/apply/destroy.
modules/iam	Contains service accounts and IAM bindings needed for workloads.	Evaluated during plan/apply/destroy.
outputs.tf	Exposes useful values such as cluster name, Cloud SQL instance name/connection information, or environment identifiers for downstream automation.	Calculated/displayed by plan/apply; useful to CI/Argo integration.

PART 5 - PARALLEL VIEW: LIFECYCLE VS OUR PIPELINE

Terraform lifecycle	Generic meaning	Our pipeline / use case
Configuration	Describe desired infrastructure.	.tf files define ephemeral VPC, GKE, Cloud SQL, IAM, Workload Identity.
init	Prepare providers, modules, and backend.	GitHub Actions downloads Google provider, initializes modules and remote state.
validate	Check complete configuration.	Pipeline catches Terraform configuration errors before infrastructure changes.
plan	Calculate desired-vs-current changes and dependencies.	Terraform determines whether PR environment needs VPC/GKE/Cloud SQL/IAM creation or updates.
apply	Make planned infrastructure changes.	Google provider calls GCP APIs to create/update resources.
state	Persist resource mapping and attributes.	Remote state records the PR environment so later runs know what already exists.
Argo CD	Outside Terraform core lifecycle.	Argo CD deploys/reconciles Spring Boot Kubernetes resources after infrastructure is ready.
Cloud SQL Proxy	Application connectivity layer.	Proxy runs as a sidecar and provides the connection path from Spring Boot to Cloud SQL.
Liquibase	Database schema lifecycle, separate from Terraform.	Liquibase creates/updates application tables and schema in Cloud SQL.
destroy	Remove Terraform-managed infrastructure.	Ephemeral GKE/Cloud SQL/network/IAM resources are removed when the environment is retired.

END-TO-END

```
Developer PR -> GitHub Actions
-> Terraform init -> providers/modules/backend ready
-> Terraform validate -> configuration checked
-> Terraform plan -> variables + .tf files + state + GCP data -> dependency graph -> plan
-> Terraform apply -> Google Provider -> GCP APIs -> VPC + GKE + Cloud SQL + IAM + Workload Identity
-> Remote state updated
-> Argo CD -> Spring Boot deployed to GKE
-> Cloud SQL Auth Proxy sidecar -> Cloud SQL PostgreSQL
-> Liquibase -> database schema migrations
-> PR closed -> Terraform destroy -> ephemeral infrastructure removed
```

Remember: Terraform file names organize the configuration; they are not separate lifecycle stages. The lifecycle operates on the complete configuration. The most important files for understanding our pipeline are providers.tf, versions.tf, backend.tf, main.tf, variables.tf, tfvars, module files, outputs.tf, and remote state.